

FluNet Data Audit: Japan

1. Project Overview

This project investigates the relationship between ambient temperature and influenza activity in Japan using WHO FluNet surveillance data.

The current objective is limited to inspecting the available FluNet file and understanding its structure before defining influenza activity. A proposed definition is:

$$\text{Influenza Positivity Rate} = \frac{\text{Influenza Positive Specimens}}{\text{Total Specimens Tested}} \times 100$$

This formula is **not finalized** here. The numerator and denominator must first be matched to clearly documented, sufficiently complete FluNet columns. This notebook does not perform correlation, ACF/CCF, regression, forecasting, hypothesis testing, weather integration, or other advanced analysis.

2. Import Libraries

```
In [1]: from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

3. File Configuration

The source file is read without modification.

```
In [2]: DATA_PATH = Path("dataset/VIW_FNT.csv")
DATA_PATH
```

```
Out[2]: PosixPath('dataset/VIW_FNT.csv')
```

4. Load the FluNet Dataset

```
In [3]: df = pd.read_csv(DATA_PATH, low_memory=False)
print(f"Dataset shape: {df.shape[0]:,} rows × {df.shape[1]:,} columns")
```

```
Dataset shape: 187,331 rows × 53 columns
```

```
In [4]: df.head(10)
```

Out [4]:

	WHOREGION	FLUSEASON	HEMISPHERE	ITZ	COUNTRY_CODE	COU
0	AMR	YR	NH	FLU_CNT_AMC		ABW
1	AMR	YR	NH	FLU_CNT_AMC		ABW
2	AMR	YR	NH	FLU_CNT_AMC		ABW
3	AMR	YR	NH	FLU_CNT_AMC		ABW
4	AMR	YR	NH	FLU_CNT_AMC		ABW
5	AMR	YR	NH	FLU_CNT_AMC		ABW
6	AMR	YR	NH	FLU_CNT_AMC		ABW
7	AMR	YR	NH	FLU_CNT_AMC		ABW
8	AMR	YR	NH	FLU_CNT_AMC		ABW
9	AMR	YR	NH	FLU_CNT_AMC		ABW

10 rows × 53 columns

In [5]: `df.tail(10)`

Out [5]:

	WHOREGION	FLUSEASON	HEMISPHERE	ITZ	COUNTRY_CODE	COU
187321	AFR	YR	SH	FLU_EST_AFR		ZWE
187322	AFR	YR	SH	FLU_EST_AFR		ZWE
187323	AFR	YR	SH	FLU_EST_AFR		ZWE
187324	AFR	YR	SH	FLU_EST_AFR		ZWE
187325	AFR	YR	SH	FLU_EST_AFR		ZWE
187326	AFR	YR	SH	FLU_EST_AFR		ZWE
187327	AFR	YR	SH	FLU_EST_AFR		ZWE
187328	AFR	YR	SH	FLU_EST_AFR		ZWE
187329	AFR	YR	SH	FLU_EST_AFR		ZWE
187330	AFR	YR	SH	FLU_EST_AFR		ZWE

10 rows × 53 columns

5. Dataset Structure

In [6]: `print("Column names:")
print(df.columns.tolist())`

Column names:

```
['WHOREGION', 'FLUSEASON', 'HEMISPHERE', 'ITZ', 'COUNTRY_CODE', 'COUNTRY_A  
REA_TERRITORY', 'ISO_WEEKSTARTDATE', 'ISO_YEAR', 'ISO_WEEK', 'MMWR_WEEKSTA  
RTDATE', 'MMWR_YEAR', 'MMWR_WEEK', 'ORIGIN_SOURCE', 'SPEC_PROCESSED_NB',  
'SPEC_RECEIVED_NB', 'AH1N12009', 'AH1', 'AH3', 'AH5', 'AH7N9', 'ANOTSUBTYP  
ED', 'ANOTSUBTYPABLE', 'AOTHER_SUBTYPE', 'AOTHER_SUBTYPE_DETAILS', 'INF_  
A', 'BVIC_2DEL', 'BVIC_3DEL', 'BVIC_NODEL', 'BVIC_DELUNK', 'BYAM', 'BNOTDE  
TERMINED', 'INF_B', 'INF_ALL', 'INF_NEGATIVE', 'ILI_ACTIVITY', 'ADENO', 'B  
OCA', 'HUMAN_CORONA', 'METAPNEUMO', 'PARAINFLUENZA', 'RHINO', 'RSV_PROCESS  
ED', 'RSV', 'OTHERRESPVIRUS', 'OTHER_RESPVIRUS_DETAILS', 'LAB_RESULT_COMME  
NT', 'WCR_COMMENT', 'IS02', 'ISOYW', 'MMWRYW', 'PSOURCE_SUBTYPE_INF', 'PSO  
URCE_PPOS_INF', 'PSOURCE_RSV']
```

```
In [7]: df.dtypes.to_frame(name="data_type")
```

Out [7]:

	data_type
WHOREGION	str
FLUSEASON	str
HEMISPHERE	str
ITZ	str
COUNTRY_CODE	str
COUNTRY_AREA_TERRITORY	str
ISO_WEEKSTARTDATE	str
ISO_YEAR	int64
ISO_WEEK	int64
MMWR_WEEKSTARTDATE	str
MMWR_YEAR	int64
MMWR_WEEK	int64
ORIGIN_SOURCE	str
SPEC_PROCESSED_NB	float64
SPEC_RECEIVED_NB	float64
AH1N12009	float64
AH1	float64
AH3	float64
AH5	float64
AH7N9	float64
ANOTSUBTYPED	float64
ANOTSUBTYPABLE	float64
AOTHER_SUBTYPE	float64
AOTHER_SUBTYPE_DETAILS	str
INF_A	float64
BVIC_2DEL	float64
BVIC_3DEL	float64
BVIC_NODEL	float64
BVIC_DELUNK	float64
BYAM	float64
BNOTDETERMINED	float64
INF_B	float64
INF_ALL	float64
INF_NEGATIVE	float64

	data_type
ILI_ACTIVITY	float64
ADENO	float64
BOCA	float64
HUMAN_CORONA	float64
METAPNEUMO	float64
PARAINFLUENZA	float64
RHINO	float64
RSV_PROCESSED	float64
RSV	float64
OTHERRESPVIRUS	float64
OTHER_RESPVIRUS_DETAILS	str
LAB_RESULT_COMMENT	str
WCR_COMMENT	str
ISO2	str
ISOYW	int64
MMWRYW	int64
PSOURCE_SUBTYPE_INF	float64
PSOURCE_PPOS_INF	float64
PSOURCE_RSV	float64

```
In [8]: df.info()
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 187331 entries, 0 to 187330
```

```
Data columns (total 53 columns):
```

#	Column	Non-Null Count	Dtype
0	WHOREGION	187331 non-null	str
1	FLUSEASON	187331 non-null	str
2	HEMISPHERE	187331 non-null	str
3	ITZ	187331 non-null	str
4	COUNTRY_CODE	187331 non-null	str
5	COUNTRY_AREA_TERRITORY	187331 non-null	str
6	ISO_WEEKSTARTDATE	187331 non-null	str
7	ISO_YEAR	187331 non-null	int64
8	ISO_WEEK	187331 non-null	int64
9	MMWR_WEEKSTARTDATE	187331 non-null	str
10	MMWR_YEAR	187331 non-null	int64
11	MMWR_WEEK	187331 non-null	int64
12	ORIGIN_SOURCE	187331 non-null	str
13	SPEC_PROCESSED_NB	174402 non-null	float64
14	SPEC_RECEIVED_NB	65555 non-null	float64
15	AH1N12009	139424 non-null	float64
16	AH1	101025 non-null	float64
17	AH3	156431 non-null	float64
18	AH5	60618 non-null	float64
19	AH7N9	25369 non-null	float64
20	ANOTSUBTYPED	150819 non-null	float64
21	ANOTSUBTYPABLE	19740 non-null	float64
22	AOTHER_SUBTYPE	48851 non-null	float64
23	AOTHER_SUBTYPE_DETAILS	7611 non-null	str
24	INF_A	162187 non-null	float64
25	BVIC_2DEL	10289 non-null	float64
26	BVIC_3DEL	9124 non-null	float64
27	BVIC_NODEL	124823 non-null	float64
28	BVIC_DELUNK	9524 non-null	float64
29	BYAM	120568 non-null	float64
30	BNOTDETERMINED	153822 non-null	float64
31	INF_B	164713 non-null	float64
32	INF_ALL	104524 non-null	float64
33	INF_NEGATIVE	75475 non-null	float64
34	ILI_ACTIVITY	73479 non-null	float64
35	ADENO	28477 non-null	float64
36	BOCA	21112 non-null	float64
37	HUMAN_CORONA	20578 non-null	float64
38	METAPNEUMO	26147 non-null	float64
39	PARAINFLUENZA	27701 non-null	float64
40	RHINO	24518 non-null	float64
41	RSV_PROCESSED	35771 non-null	float64
42	RSV	90887 non-null	float64
43	OTHERRESPVIRUS	24299 non-null	float64
44	OTHER_RESPVIRUS_DETAILS	25191 non-null	str
45	LAB_RESULT_COMMENT	5053 non-null	str
46	WCR_COMMENT	19296 non-null	str
47	IS02	187152 non-null	str
48	ISOYW	187331 non-null	int64
49	MMWRYW	187331 non-null	int64
50	PSOURCE_SUBTYPE_INF	79332 non-null	float64
51	PSOURCE_PPOS_INF	79332 non-null	float64
52	PSOURCE_RSV	78593 non-null	float64

```
dtypes: float64(33), int64(6), str(14)
```

```
memory usage: 75.7 MB
```

```
In [9]: structure_summary = pd.DataFrame({
    "column_name": df.columns,
    "data_type": df.dtypes.astype(str).values,
    "missing_values": df.isna().sum().values,
    "unique_values": df.nunique(dropna=True).values,
})
structure_summary
```

Out [9]:

	column_name	data_type	missing_values	unique_values
0	WHOREGION	str	0	6
1	FLUSEASON	str	0	3
2	HEMISPHERE	str	0	2
3	ITZ	str	0	18
4	COUNTRY_CODE	str	0	189
5	COUNTRY_AREA_TERRITORY	str	0	189
6	ISO_WEEKSTARTDATE	str	0	1624
7	ISO_YEAR	int64	0	32
8	ISO_WEEK	int64	0	53
9	MMWR_WEEKSTARTDATE	str	0	1624
10	MMWR_YEAR	int64	0	32
11	MMWR_WEEK	int64	0	53
12	ORIGIN_SOURCE	str	0	3
13	SPEC_PROCESSED_NB	float64	12929	7625
14	SPEC_RECEIVED_NB	float64	121776	5056
15	AH1N12009	float64	47907	1076
16	AH1	float64	86306	235
17	AH3	float64	30900	1031
18	AH5	float64	126713	15
19	AH7N9	float64	161962	7
20	ANOTSUBTYPED	float64	36512	1638
21	ANOTSUBTYPABLE	float64	167591	52
22	AOTHER_SUBTYPE	float64	138480	24
23	AOTHER_SUBTYPE_DETAILS	str	179720	79
24	INF_A	float64	25144	2274
25	BVIC_2DEL	float64	177042	38
26	BVIC_3DEL	float64	178207	6
27	BVIC_NODEL	float64	62508	421
28	BVIC_DELUNK	float64	177807	85
29	BYAM	float64	66763	275
30	BNOTDETERMINED	float64	33509	998
31	INF_B	float64	22618	1164
32	INF_ALL	float64	82807	2581
33	INF_NEGATIVE	float64	111856	5238

	column_name	data_type	missing_values	unique_values
34	ILI_ACTIVITY	float64	113852	7
35	ADENO	float64	158854	226
36	BOCA	float64	166219	59
37	HUMAN_CORONA	float64	166753	286
38	METAPNEUMO	float64	161184	361
39	PARAINFLUENZA	float64	159630	374
40	RHINO	float64	162813	636
41	RSV_PROCESSED	float64	151560	3860
42	RSV	float64	96444	1083
43	OTHERRESPVIRUS	float64	163032	407
44	OTHER_RESPVIRUS_DETAILS	str	162140	5666
45	LAB_RESULT_COMMENT	str	182278	1443
46	WCR_COMMENT	str	168035	11590
47	ISO2	str	179	188
48	ISOYW	int64	0	1624
49	MMWRYW	int64	0	1624
50	PSOURCE_SUBTYPE_INF	float64	107999	1
51	PSOURCE_PPOS_INF	float64	107999	1
52	PSOURCE_RSV	float64	108738	1

6. Identify Japan Data

`COUNTRY_AREA_TERRITORY` is the explicit country/location name field. `COUNTRY_CODE` and `ISO2` provide corresponding codes. The checks below establish whether the file contains multiple countries and how Japan is represented. The original `df` is retained unchanged.

```
In [10]: country_columns = ["COUNTRY_CODE", "COUNTRY_AREA_TERRITORY", "ISO2"]
country_lookup = (
    df[country_columns]
    .drop_duplicates()
    .sort_values("COUNTRY_AREA_TERRITORY")
    .reset_index(drop=True)
)

print(f"Number of countries/areas: {df['COUNTRY_AREA_TERRITORY'].nunique(
country_lookup
```

Number of countries/areas: 189

Out [10]:

	COUNTRY_CODE	COUNTRY_AREA_TERRITORY	ISO2
0	AFG	Afghanistan	AF
1	ALB	Albania	AL
2	DZA	Algeria	DZ
3	AGO	Angola	AO
4	AIA	Anguilla	AI
...	...	...	...
184	VNM	Viet Nam	VN
185	YEM	Yemen	YE
186	ZMB	Zambia	ZM
187	ZWE	Zimbabwe	ZW
188	PSE	occupied Palestinian territory, including east...	PS

189 rows × 3 columns

In [11]:

```
japan_values = country_lookup[
    country_lookup["COUNTRY_AREA_TERRITORY"].str.contains("Japan", case=F
]
japan_values
```

Out [11]:

	COUNTRY_CODE	COUNTRY_AREA_TERRITORY	ISO2
81	JPN	Japan	JP

In [12]:

```
df_japan = df[df["COUNTRY_AREA_TERRITORY"] == "Japan"].copy()
print(f"Japan subset shape: {df_japan.shape[0]:,} rows × {df_japan.shape[1]:,} columns")
df_japan[["COUNTRY_CODE", "COUNTRY_AREA_TERRITORY", "ISO2", "ORIGIN_SOURCE"]]
```

Japan subset shape: 1,522 rows × 53 columns

Out [12]:

	COUNTRY_CODE	COUNTRY_AREA_TERRITORY	ISO2	ORIGIN_SOURCE
84839	JPN	Japan	JP	SENTINEL

7. Identify Relevant FluNet Variables

Based on exact column names and observed values, these are the candidate fields. The names strongly suggest their roles, but column labels alone are not a substitute for the official FluNet data dictionary.

Concept	FluNet Column	Notes
Year	ISO_YEAR	ISO epidemiological year; paired with ISO_WEEK and ISO_WEEKSTARTDATE
Week	ISO_WEEK	ISO epidemiological week (1–53)

Concept	FluNet Column	Notes
Influenza A positive	INF_A	Candidate count of influenza A-positive specimens
Influenza B positive	INF_B	Candidate count of influenza B-positive specimens
Total positive	INF_ALL	Candidate total influenza-positive count; checked against <code>INF_A + INF_B</code> below
Total tested	SPEC_PROCESSED_NB	Candidate number of specimens processed; meaning and sparse Japan coverage require verification

`SPEC_RECEIVED_NB` is also inspected because its name concerns specimens, but it is entirely missing for Japan and is not treated as the denominator.

`INF_NEGATIVE` may help cross-check counts, but it is not assumed to be a tested-total field.

```
In [13]: candidate_columns = [
            "ISO_YEAR", "ISO_WEEK", "ISO_WEEKSTARTDATE",
            "INF_A", "INF_B", "INF_ALL",
            "SPEC_PROCESSED_NB", "SPEC_RECEIVED_NB", "INF_NEGATIVE",
        ]
df_japan[candidate_columns].head(12)
```

```
Out[13]:
```

	ISO_YEAR	ISO_WEEK	ISO_WEEKSTARTDATE	INF_A	INF_B	INF_ALL	SPEC
84839	1996	41	1996-10-07	1.0	0.0	1.0	
84840	1996	42	1996-10-14	2.0	0.0	2.0	
84841	1996	43	1996-10-21	1.0	0.0	1.0	
84842	1996	44	1996-10-28	4.0	0.0	4.0	
84843	1996	45	1996-11-04	11.0	0.0	11.0	
84844	1996	46	1996-11-11	12.0	0.0	12.0	
84845	1996	47	1996-11-18	25.0	1.0	26.0	
84846	1996	48	1996-11-25	63.0	2.0	65.0	
84847	1996	49	1996-12-02	190.0	4.0	194.0	
84848	1996	50	1996-12-09	433.0	3.0	436.0	
84849	1996	51	1996-12-16	401.0	10.0	411.0	
84850	1996	52	1996-12-23	269.0	1.0	270.0	

```
In [14]: sample_values = pd.DataFrame({
            "column": candidate_columns,
            "sample_non_missing_values": [
                df_japan[column].dropna().drop_duplicates().head(8).tolist()
                for column in candidate_columns
            ],
        })
sample_values
```

Out [14]:

	column	sample_non_missing_values
0	ISO_YEAR	[1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003]
1	ISO_WEEK	[41, 42, 43, 44, 45, 46, 47, 48]
2	ISO_WEEKSTARTDATE	[1996-10-07, 1996-10-14, 1996-10-21, 1996-10-2...
3	INF_A	[1.0, 2.0, 4.0, 11.0, 12.0, 25.0, 63.0, 190.0]
4	INF_B	[0.0, 1.0, 2.0, 4.0, 3.0, 10.0, 8.0, 19.0]
5	INF_ALL	[1.0, 2.0, 4.0, 11.0, 12.0, 26.0, 65.0, 194.0]
6	SPEC_PROCESSED_NB	[1148.0, 1144.0, 1167.0, 966.0, 828.0, 1150.0,...
7	SPEC_RECEIVED_NB	[]
8	INF_NEGATIVE	[2.0, 6.0, 7.0, 4.0, 0.0, 8.0, 9.0, 13.0]

8. Time Coverage

```
In [15]: year_summary = pd.Series({
    "minimum_year": df_japan["ISO_YEAR"].min(),
    "maximum_year": df_japan["ISO_YEAR"].max(),
    "number_of_unique_years": df_japan["ISO_YEAR"].nunique(),
})
year_summary
```

```
Out [15]: minimum_year      1996
maximum_year      2026
number_of_unique_years      31
dtype: int64
```

```
In [16]: records_per_year = (
    df_japan.groupby("ISO_YEAR")
    .size()
    .rename("number_of_records")
    .to_frame()
)
records_per_year
```

Out [16]:

number_of_records

ISO_YEAR	
1996	12
1997	20
1998	51
1999	52
2000	51
2001	52
2002	52
2003	52
2004	53
2005	52
2006	52
2007	52
2008	52
2009	53
2010	52
2011	52
2012	52
2013	52
2014	52
2015	53
2016	52
2017	52
2018	52
2019	52
2020	53
2021	52
2022	52
2023	52
2024	52
2025	52
2026	32

```
In [17]: week_summary = pd.Series({  
        "minimum_week": df_japan["ISO_WEEK"].min(),
```

```

    "maximum_week": df_japan["ISO_WEEK"].max(),
    "number_of_unique_weeks": df_japan["ISO_WEEK"].nunique(),
})
print(week_summary)
print("Unique epidemiological weeks:")
print(sorted(df_japan["ISO_WEEK"].unique()))

```

```

minimum_week          1
maximum_week          53
number_of_unique_weeks  53
dtype: int64
Unique epidemiological weeks:
[np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(
6), np.int64(7), np.int64(8), np.int64(9), np.int64(10), np.int64(11), np.i
nt64(12), np.int64(13), np.int64(14), np.int64(15), np.int64(16), np.int64(17), np.int64(18), np.int64(19), np.int64(20), np.int64(21), np.int64(22), np.int64(23), np.int64(24), np.int64(25), np.int64(26), np.int64(27), np.int64(28), np.int64(29), np.int64(30), np.int64(31), np.int64(32), np.int64(33), np.int64(34), np.int64(35), np.int64(36), np.int64(37), np.int64(38), np.int64(39), np.int64(40), np.int64(41), np.int64(42), np.int64(43), np.int64(44), np.int64(45), np.int64(46), np.int64(47), np.int64(48), np.int64(49), np.int64(50), np.int64(51), np.int64(52), np.int64(53)]

```

9. Year–Week Structure

An ISO year–week identifier is created in the copied Japan DataFrame. Duplicate counts below use `ISO_YEAR` and `ISO_WEEK`; no records are aggregated.

```

In [18]: df_japan["year_week"] = (
    df_japan["ISO_YEAR"].astype(str)
    + "-W"
    + df_japan["ISO_WEEK"].astype(str).str.zfill(2)
)

duplicate_mask = df_japan.duplicated(["ISO_YEAR", "ISO_WEEK"], keep=False)
year_week_summary = pd.Series({
    "total_rows": len(df_japan),
    "unique_year_week_combinations": df_japan[["ISO_YEAR", "ISO_WEEK"]].d
    "rows_in_duplicated_year_weeks": int(duplicate_mask.sum()),
    "extra_duplicated_year_week_rows": int(
        df_japan.duplicated(["ISO_YEAR", "ISO_WEEK"]).sum()
    ),
})
year_week_summary

```

```

Out[18]: total_rows          1522
unique_year_week_combinations  1522
rows_in_duplicated_year_weeks      0
extra_duplicated_year_week_rows      0
dtype: int64

```

```

In [19]: duplicated_records = df_japan.loc[duplicate_mask].sort_values(
    ["ISO_YEAR", "ISO_WEEK"]
)

if duplicated_records.empty:
    print("No duplicated ISO year–week records were found for Japan.")
else:

```

```

display(duplicated_records)
differing_columns = [
    column for column in duplicated_records.columns
    if duplicated_records.groupby(["ISO_YEAR", "ISO_WEEK"])[column]
    .nunique(dropna=False).gt(1).any()
]
print("Columns that differ within at least one duplicated year-week:")
print(differing_columns)

```

No duplicated ISO year-week records were found for Japan.

10. Missing Values in Relevant Variables

```

In [20]: relevant_columns = [
    "ISO_YEAR", "ISO_WEEK", "INF_A", "INF_B", "INF_ALL",
    "SPEC_PROCESSED_NB", "SPEC_RECEIVED_NB", "INF_NEGATIVE",
]

missing_summary = pd.DataFrame({
    "missing_values": df_japan[relevant_columns].isna().sum(),
    "missing_percent": df_japan[relevant_columns].isna().mean().mul(100).
})
missing_summary

```

```

Out[20]:

```

	missing_values	missing_percent
ISO_YEAR	0	0.00
ISO_WEEK	0	0.00
INF_A	20	1.31
INF_B	18	1.18
INF_ALL	246	16.16
SPEC_PROCESSED_NB	1452	95.40
SPEC_RECEIVED_NB	1522	100.00
INF_NEGATIVE	1186	77.92

```

In [21]: # Coverage by year helps reveal whether missingness is concentrated in ti
non_missing_by_year = df_japan.groupby("ISO_YEAR")[[
    "INF_A", "INF_B", "INF_ALL", "SPEC_PROCESSED_NB", "INF_NEGATIVE"
]].count()
non_missing_by_year

```

Out[21]:

	INF_A	INF_B	INF_ALL	SPEC_PROCESSED_NB	INF_NEGATIVE
ISO_YEAR					
1996	12	12	12	0	0
1997	17	15	17	0	0
1998	40	38	26	0	0
1999	52	52	33	0	0
2000	51	51	29	0	0
2001	52	52	41	0	0
2002	52	52	36	0	0
2003	52	52	38	0	0
2004	53	53	44	0	0
2005	52	52	49	0	0
2006	52	52	46	0	0
2007	52	52	49	0	0
2008	52	52	47	0	0
2009	53	53	53	0	0
2010	52	52	52	0	0
2011	52	52	51	0	0
2012	52	52	52	0	0
2013	52	52	52	0	0
2014	52	52	51	0	0
2015	53	53	53	0	0
2016	52	52	52	0	52
2017	52	52	52	0	51
2018	52	52	52	0	52
2019	52	52	52	0	52
2020	53	53	19	0	53
2021	52	52	6	0	52
2022	47	52	24	0	24
2023	52	52	52	0	0
2024	52	52	52	0	0
2025	52	52	52	38	0
2026	31	32	32	32	0

11. Check Numerical Validity


```
In [22]: numeric_influenza_columns = [
        "INF_A", "INF_B", "INF_ALL", "SPEC_PROCESSED_NB", "INF_NEGATIVE"
    ]

    numeric_validity = pd.DataFrame({
        "minimum": df_japan[numeric_influenza_columns].min(),
        "maximum": df_japan[numeric_influenza_columns].max(),
        "mean": df_japan[numeric_influenza_columns].mean(),
        "median": df_japan[numeric_influenza_columns].median(),
        "negative_values": (df_japan[numeric_influenza_columns] < 0).sum(),
        "zero_values": (df_japan[numeric_influenza_columns] == 0).sum(),
    })
    numeric_validity
```

```
Out [22]:
```

	minimum	maximum	mean	median	negative_values
INF_A	0.0	1656.0	112.820240	16.0	0
INF_B	0.0	576.0	31.910904	3.0	0
INF_ALL	1.0	1656.0	170.415361	48.5	0
SPEC_PROCESSED_NB	54.0	1273.0	905.900000	1038.5	0
INF_NEGATIVE	0.0	6599.0	828.514881	49.0	0

```
In [23]: processed_checks = pd.Series({
        "missing_values": df_japan["SPEC_PROCESSED_NB"].isna().sum(),
        "zero_values": df_japan["SPEC_PROCESSED_NB"].eq(0).sum(),
        "negative_values": df_japan["SPEC_PROCESSED_NB"].lt(0).sum(),
    })
    processed_checks
```

```
Out [23]: missing_values    1452
zero_values                0
negative_values            0
dtype: int64
```

12. Inspect Influenza Positive Variables

The following comparison is restricted to rows where `INF_A`, `INF_B`, and `INF_ALL` are all present. Missing values are not converted to zero, and the original columns are unchanged.

```
In [24]: positive_comparison = df_japan.loc[
        df_japan[["INF_A", "INF_B", "INF_ALL"]].notna().all(axis=1),
        ["ISO_YEAR", "ISO_WEEK", "INF_A", "INF_B", "INF_ALL"],
    ].copy()

    positive_comparison["INF_A_plus_INF_B"] = (
        positive_comparison["INF_A"] + positive_comparison["INF_B"]
    )
    positive_comparison["matches_INF_ALL"] = (
        positive_comparison["INF_A_plus_INF_B"] == positive_comparison["INF_A"]
    )

    comparison_summary = pd.Series({
```

```

    "comparable_rows": len(positive_comparison),
    "matching_rows": positive_comparison["matches_INF_ALL"].sum(),
    "non_matching_rows": (~positive_comparison["matches_INF_ALL"]).sum(),
})
comparison_summary

```

```

Out[24]: comparable_rows    1267
         matching_rows    1267
         non_matching_rows      0
         dtype: int64

```

```

In [25]: non_matching_positive_rows = positive_comparison.loc[
         ~positive_comparison["matches_INF_ALL"]
         ]

if non_matching_positive_rows.empty:
    print("No non-matching rows were found among rows with all three coun
else:
    display(non_matching_positive_rows.head(10))

```

No non-matching rows were found among rows with all three counts present.

13. Candidate Positivity Rate Variables

Candidate Numerator

`INF_ALL` appears to be the most direct candidate for **Influenza Positive Specimens**. Where all three fields are available, it can be checked against `INF_A + INF_B`. However, `INF_ALL` is missing in some Japan rows. Using `INF_A + INF_B` to fill those gaps would be a preprocessing decision and is not done in this audit.

Candidate Denominator

`SPEC_PROCESSED_NB` appears to be the most direct candidate for **Total Specimens Tested** because it records the number of specimens processed. It is preferable to `SPEC_RECEIVED_NB`, which is entirely missing for Japan. Nevertheless, `SPEC_PROCESSED_NB` is available only in a small, recent portion of the Japan series, so its exact definition and compatibility across time require verification against WHO FluNet documentation or another denominator source.

`INF_NEGATIVE` is not assumed to be the denominator. A possible derived total such as `INF_ALL + INF_NEGATIVE` would require a documented confirmation that the counts share the same testing scope.

14. Preliminary Positivity Rate Feasibility Check

For a limited sanity check only, positivity is calculated where both candidate fields are present and `SPEC_PROCESSED_NB > 0`. Weeks with zero or missing processed specimens receive no calculated rate. The raw DataFrame remains unchanged.

```
In [26]: rate_check = df_japan.loc[
    df_japan["INF_ALL"].notna() & df_japan["SPEC_PROCESSED_NB"].gt(0),
    ["ISO_YEAR", "ISO_WEEK", "ISO_WEEKSTARTDATE", "INF_ALL", "SPEC_PROCESSED_NB"].copy()

rate_check["positivity_rate"] = (
    rate_check["INF_ALL"] / rate_check["SPEC_PROCESSED_NB"] * 100
)

rate_summary = pd.Series({
    "eligible_rows": len(rate_check),
    "minimum_percent": rate_check["positivity_rate"].min(),
    "maximum_percent": rate_check["positivity_rate"].max(),
    "mean_percent": rate_check["positivity_rate"].mean(),
    "median_percent": rate_check["positivity_rate"].median(),
    "values_outside_0_to_100": (
        ~rate_check["positivity_rate"].between(0, 100)
    ).sum(),
})
rate_summary
```

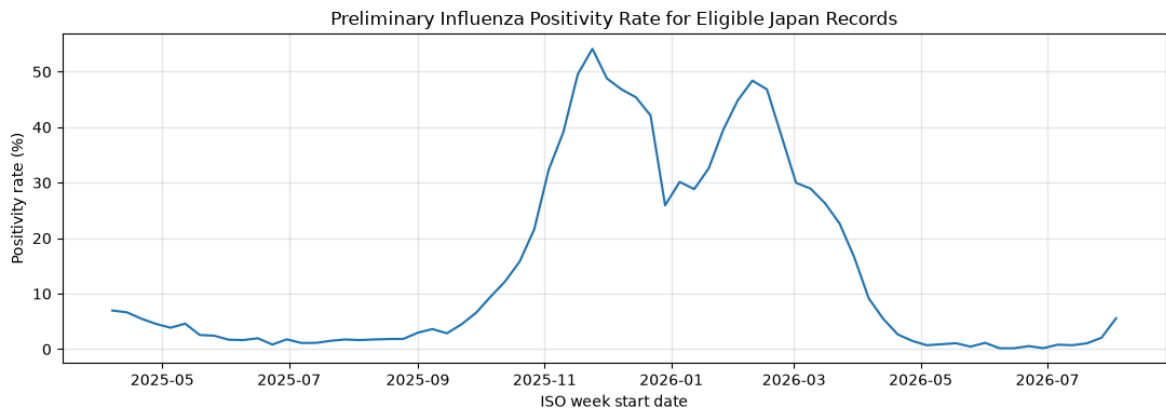
```
Out[26]: eligible_rows      70.000000
minimum_percent      0.165017
maximum_percent      54.136691
mean_percent      14.139230
median_percent      4.516239
values_outside_0_to_100      0.000000
dtype: float64
```

15. Simple Preliminary Visualization

This plot shows only rows eligible for the preliminary calculation. It is a visual data check, not a statistical interpretation or the final analysis series.

```
In [27]: rate_check["week_start_date"] = pd.to_datetime(rate_check["ISO_WEEKSTARTDATE"])
rate_check = rate_check.sort_values("week_start_date")

fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(rate_check["week_start_date"], rate_check["positivity_rate"], linecolor='red')
ax.set_title("Preliminary Influenza Positivity Rate for Eligible Japan Regions")
ax.set_xlabel("ISO week start date")
ax.set_ylabel("Positivity rate (%)")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



FluNet Data Audit Summary

A. Dataset

- **File used:** `dataset/VIW_FNT.csv` .
- **Geographic scope:** The file contains multiple countries/areas (189 distinct `COUNTRY_AREA_TERRITORY` values). Japan is represented as `COUNTRY_AREA_TERRITORY = "Japan"` , `COUNTRY_CODE = "JPN"` , and `ISO2 = "JP"` .
- **Japan time period:** ISO year 1996 through 2026. The first and latest years are partial in this file (12 records in 1996 and 32 in 2026).

B. Data Structure

- The Japan subset contains 1,522 records and 1,522 distinct ISO year-week combinations.
- Each Japan row therefore represents one unique epidemiological week in the available file.
- No duplicated Japan ISO year-week records were found.
- All Japan records have `ORIGIN_SOURCE = "SENTINEL"` .

C. Influenza Activity Definition

- **Candidate numerator:** `INF_ALL` , supported by its exact equality to `INF_A + INF_B` in all 1,267 rows where all three values are present.
- **Candidate denominator:** `SPEC_PROCESSED_NB` , based on its name and intended role as specimens processed. Its definition should be confirmed in official metadata before final use.

D. Data Quality

- `ISO_YEAR` and `ISO_WEEK` have no missing values.
- `INF_A` , `INF_B` , and `INF_ALL` have 20, 18, and 246 missing values, respectively.
- `SPEC_PROCESSED_NB` has 1,452 missing values (95.40%) and is populated only for 70 records in 2025–2026.

- `SPEC_RECEIVED_NB` is missing for every Japan record.
- There are no negative values in the inspected influenza count fields.
`SPEC_PROCESSED_NB` has no zero or negative values among its 70 observed records.
- Among comparable records, no inconsistency was found between `INF_A + INF_B` and `INF_ALL`.
- The preliminary rate has 70 eligible records, all within 0–100%; this validates arithmetic only for the small available subset.

E. Next Decision

Conclusion: Requires further verification.

The data support a plausible numerator, have a clean unique weekly structure, and show internally consistent positive counts. However, the candidate denominator is missing for 95.40% of Japan weeks and appears only in 2025–2026. Before defining influenza activity as weekly positivity rate for the full study period, confirm the FluNet field definitions and determine whether a documented, comparable denominator is available for earlier years. Any construction from `INF_ALL + INF_NEGATIVE`, or any substitution of `INF_A + INF_B` for missing `INF_ALL`, should be treated as a separate, justified preprocessing decision.